

Delivering & Operating Systems

The core of Part VIII — shipping code safely and running it reliably.

46 CI/CD & Deployment Strategies — shipping changes safely and often

47 Infrastructure as Code — managing infrastructure like software

48 Cloud & Multi-Region Architecture — running close to users, surviving failures

49 Cost Optimization — designing for the bill, not just the load

50 Observability — logs, metrics, and traces

Chapters 46-47 automate delivery and infrastructure; 48-50 run it across regions, within budget, and with visibility.

CHAPTER 46

CI/CD & Deployment Strategies

Part VIII · Infrastructure & Operations — shipping changes safely and often.

LEARNING OBJECTIVES

- Define continuous integration, delivery, and deployment.
- Describe a typical pipeline from commit to production.
- Compare rolling, blue-green, and canary deployments.
- Explain feature flags and fast rollback.

REAL-WORLD ANALOGY — AN ASSEMBLY LINE WITH QUALITY GATES

A modern factory doesn't hand-build one giant product a year; it runs a continuous line where every part passes automated quality checks, and a finished unit can roll off at any time. **CI/CD** is that line for software: small changes flow through automated tests and checks, ready to ship at any moment.

46.1 The problem

Manual, infrequent, big-bang releases are slow and risky — large batches of untested change fail spectacularly. The fix is to **automate** building, testing, and releasing, and to ship **small changes frequently** so each is low-risk and easy to diagnose.

46.2 CI, CD, and CD

Continuous Integration: developers merge small changes often, and every merge triggers an automated build and test suite — catching breakage immediately. **Continuous Delivery:** every passing build is automatically made **ready** to release (a human clicks deploy). **Continuous Deployment:** passing builds go to production **automatically**, no human step.



46.3 Deployment strategies

Strategy	How it works
Rolling	Replace instances a few at a time. Simple, no extra capacity; mixed versions run briefly.
Blue-green	Run two full environments; switch traffic from old (blue) to new (green) at once. Instant rollback, but double the resources.
Canary	Release to a small % of users first, watch metrics, then ramp up. Limits blast radius; needs good monitoring.

46.4 Feature flags and rollback

KEY INSIGHT — SEPARATE DEPLOY FROM RELEASE

Feature flags let you **deploy** code with a feature turned off, then **release** it by flipping a flag — no redeploy, and instant off-switch if it misbehaves. Combined with a fast, automated **rollback** (and canaries to catch problems early), this makes shipping safe: the ability to undo a bad release in seconds matters more than never shipping bugs.

Common mistakes

WATCH OUT

- Big, infrequent releases that bundle many risky changes.
- A pipeline without real automated tests (a false sense of safety).
- Deploying to 100% at once with no canary and no fast rollback.
- Database migrations that aren't backward-compatible, breaking rollback.

Interview questions

1. Define CI, continuous delivery, and continuous deployment.
2. Walk through a commit-to-production pipeline.
3. Compare rolling, blue-green, and canary deployments.
4. How do feature flags separate deploy from release?
5. Why is fast rollback so important?

Hands-on exercises

1. Sketch a CI pipeline with build, test, scan, and deploy stages.
2. Choose a deployment strategy for a risky change to a payment service.
3. Wrap a new feature in a flag and describe the rollout plan.
4. Explain how to make a schema change that keeps rollback safe.

Mini project

BUILD A PIPELINE WITH A CANARY

Set up a CI/CD pipeline that builds, tests, and deploys a small app. Add a canary stage that routes 5% of traffic to the new version, checks error rate, and either promotes or rolls back. Put a new feature behind a flag and practice turning it on and off without redeploying.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE DELIVERY PIPELINE

Define your pipeline: automated tests + security scan on every merge, deploy to staging, then **canary** to production with automatic rollback on error-rate regression. Use feature flags for risky features (a new feed algorithm), and require backward-compatible migrations so rollback is always safe.

Chapter summary

- **CI** auto-builds and tests every merge; **continuous delivery** keeps builds release-ready; **continuous deployment** ships automatically.
- Deploy strategies: **rolling** (simple), **blue-green** (instant switch/rollback), **canary** (gradual, low blast radius).
- **Feature flags** separate deploy from release and provide an instant off-switch.
- Small frequent changes + **fast rollback** make shipping safe.

CHAPTER 47

Infrastructure as Code

Part VIII · Infrastructure & Operations — managing infrastructure like software.

LEARNING OBJECTIVES

- Explain the problems of manually-managed infrastructure.
- Explain declarative infrastructure as code and idempotency.
- Distinguish provisioning from configuration management.
- Explain immutable infrastructure.

REAL-WORLD ANALOGY — A BLUEPRINT, NOT A MEMORY

You wouldn't rebuild a house from one worker's memory of how they did it last time — you use a **blueprint** anyone can follow to produce the identical building. Infrastructure as code is that blueprint: your servers, networks, and databases are defined in files, versioned and repeatable, not clicked together by hand and forgotten.

47.1 The problem

Manually provisioned infrastructure — clicking in a cloud console, SSH-ing to tweak servers — is **unrepeatable, undocumented, and drifts**. Environments diverge (“works in staging”), disasters can't be rebuilt reliably, and no one knows the exact current state. **Snowflake servers** that were hand-tuned are impossible to reproduce.

47.2 Infrastructure as code

Infrastructure as Code (IaC) defines infrastructure in **declarative files** stored in version control. You describe the **desired state** (“3 servers, this network, this database”) and a tool (Terraform, CloudFormation, Pulumi) makes reality match. Benefits: repeatable, reviewable, versioned, and auditable — the same rigor as application code (code review, history, rollback).

47.3 Idempotency and declarative state

KEY INSIGHT — DECLARE THE DESTINATION, NOT THE STEPS

Good IaC is **declarative** and **idempotent**: you state the end goal, and applying it repeatedly converges to that state without duplicating anything. Run it once or ten times and you get the same result. This is exactly the “desired state” idea from Kubernetes (Chapter 45), applied to all infrastructure — and it makes changes safe to re-run.

47.4 Provisioning vs configuration

Provisioning (Terraform, CloudFormation)

Configuration management (Ansible, Chef, Puppet)

Creates the infrastructure itself — servers, networks, load balancers, databases.

Configures what runs *on* servers — packages, files, services.

47.5 Immutable infrastructure

Rather than patching live servers (which causes drift), **immutable infrastructure** rebuilds from a fresh image for every change and replaces the old servers — the container mindset (Chapter 45) applied to whole machines. No in-place mutation means no drift, and rollback is just redeploying the previous image.

Common mistakes

WATCH OUT

- Making manual changes alongside IaC, causing **drift** from the declared state.
- Not versioning IaC or its **state file** (and leaking secrets into it).
- Imperative scripts that aren't idempotent and break on re-run.
- Hand-patching “snowflake” servers instead of rebuilding immutably.

Interview questions

1. What problems does IaC solve?
2. Why prefer declarative, idempotent definitions?
3. Provisioning vs configuration management?
4. What is immutable infrastructure, and why does it prevent drift?

Hands-on exercises

1. Explain how IaC would let you rebuild a lost environment.
2. Describe drift and how declarative IaC detects and corrects it.
3. Classify tasks as provisioning or configuration.
4. Give a rollback story for immutable infrastructure.

Mini project

DEFINE INFRASTRUCTURE DECLARATIVELY

Use an IaC tool to declare a small stack (a network, a couple of servers, a load balancer). Apply it, then apply again to see idempotency (no changes). Change the desired replica count and re-apply. Then make a manual change out-of-band and watch the next apply detect and correct the drift.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — CODIFIED INFRASTRUCTURE

Define your entire environment as code: networks, Kubernetes cluster (Chapter 45), databases and replicas (Chapter 16), load balancers (Chapter 7), and DNS. Keep it in version control with reviews, treat servers as immutable, and confirm you could recreate the whole platform in a new region from these files alone (setting up Chapter 48).

Chapter summary

- Manual infrastructure is unrepeatable and drifts into **snowflakes**.
- **IaC** defines infrastructure declaratively in version control — repeatable, reviewable, auditable.
- Good IaC is **declarative and idempotent** (desired-state convergence).
- **Provisioning** creates infrastructure; **configuration management** sets up servers; **immutable infrastructure** rebuilds instead of patching.

CHAPTER 48

Cloud & Multi-Region Architecture

Part VIII · Infrastructure & Operations — running close to users, surviving failures.

LEARNING OBJECTIVES

- Explain regions and availability zones.
- Explain multi-AZ and multi-region deployments and what each protects against.
- Compare active-passive and active-active across regions.
- Explain the hard part: cross-region data and consistency.

REAL-WORLD ANALOGY — WAREHOUSES NEAR YOUR CUSTOMERS

A retailer with one central warehouse is slow to distant customers and doomed if that warehouse floods. So it builds regional warehouses close to customers (fast delivery) and multiple buildings within each region (survive a local fire). Cloud **regions** and **availability zones** are exactly this for computing.

48.1 The problem

Users are spread across the globe, and a single data center is both far from most of them (latency, Chapter 5) and a giant single point of failure (Chapter 34). Cloud providers give us **geographic distribution** to fix both.

48.2 Regions and availability zones



multiple AZs survive a data-center failure; multiple regions cut latency and survive a regional outage

A **region** is a geographic location (US-East, EU-West). Within it are multiple **availability zones (AZs)** — isolated data centers with independent power and networking, linked by fast low-latency connections.

48.3 Multi-AZ vs multi-region

Multi-AZ: spread across zones in one region — survives a data-center failure with negligible latency cost. This is the standard baseline for high availability. **Multi-region:** run in several regions — puts data close to users worldwide *and* survives an entire region going down, at much higher complexity.

48.4 Active-passive vs active-active

Active-passive	Active-active
One region serves; another is a standby, promoted on failure (failover). Simpler; some downtime and possible data loss on failover.	All regions serve traffic simultaneously (routed by geo-DNS). Lowest latency and best availability, but hardest — especially for writes.

48.5 The hard part: data

CROSS-REGION CONSISTENCY IS THE REAL CHALLENGE

Compute is easy to replicate; **data** is not. Cross-region links have tens of milliseconds of latency (Chapter 5), so synchronous replication is slow and CAP (Chapter 20) forces a choice on partitions. Active-active **writes** in multiple regions risk conflicts (multi-leader, Chapter 16) and usually mean **eventual consistency** (Chapter 21). Many systems keep writes in one primary region and serve reads locally elsewhere. **Data residency** laws add further constraints. Design data placement first — it dominates multi-region architecture.

Common mistakes

WATCH OUT

- Running in a single AZ and calling it highly available.
- Going multi-region for compute while ignoring the data problem.
- Active-active writes without a conflict-resolution strategy.
- Forgetting cross-region latency and egress cost (Chapter 49) and data-residency law.

Interview questions

1. What are regions and availability zones?
2. What does multi-AZ protect against, versus multi-region?
3. Compare active-passive and active-active.
4. Why is cross-region data the hardest part, and how do teams handle it?

Hands-on exercises

1. Decide multi-AZ or multi-region for: an internal tool; a global consumer app.
2. Explain the failover steps for an active-passive setup.
3. Describe how to serve global reads while keeping writes in one region.
4. List three costs/risks that multi-region adds.

Mini project

DESIGN A TWO-REGION DEPLOYMENT

Take your IaC (Chapter 47) and design a second region. Start active-passive: primary serves writes, secondary is a warm standby with replicated data; script a failover. Then

sketch what active-active would require for reads (geo-DNS, local read replicas) and where writes would still be centralized.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — GOING GLOBAL

Make the platform multi-AZ within each region as the HA baseline, then plan multi-region: geo-route users to the nearest region (Chapter 4 DNS), serve reads from local replicas, and keep writes in a primary region (or accept eventual consistency for feed data). Note the CAP choice per data type and the residency constraints.

Chapter summary

- **Regions** are geographic; **availability zones** are isolated data centers within a region.
- **Multi-AZ** survives a data-center failure (HA baseline); **multi-region** cuts global latency and survives a regional outage.
- **Active-passive** (standby + failover) is simpler; **active-active** is best but hardest.
- The dominant challenge is **cross-region data/consistency** — design data placement first.

CHAPTER 49

Cost Optimization

Part VIII · Infrastructure & Operations — designing for the bill, not just the load.

LEARNING OBJECTIVES

- Explain why cost is a system-design constraint.
- Identify the main cost drivers in the cloud.
- Apply pricing models and right-sizing to cut spend.
- Weigh performance and reliability against cost.

REAL-WORLD ANALOGY — A UTILITY BILL

Cloud computing is billed like electricity and water: you pay for what you use, and leaving the lights on in empty rooms adds up fast. A household that watches its meter — efficient appliances, off-peak rates, no leaks — pays a fraction of a careless one for the same comfort. Cost optimization is reading the meter for your system.

49.1 The problem

The cloud makes it trivial to spin up resources — and to **overspend**. Cost is a real design constraint, not an afterthought: an architecture that ignores the bill can be technically excellent and financially ruinous. Good engineers design for a target cost per request or per user.

49.2 The main cost drivers

Typically: **compute** (servers/containers), **storage** (data volume × how “hot” it is), **data transfer** — especially **egress** (data leaving the cloud or crossing regions), **managed services** (databases, queues), and **idle/over-provisioned** resources. Cross-region and internet egress in particular surprise teams (recall it from Chapters 19 and 48).

49.3 Levers to reduce cost

Lever	How it helps
Right-sizing	Match instance size to real usage; kill idle resources.
Autoscaling	Scale down at low load so you don’t pay for peak 24/7 (Chapter 10).
Pricing models	Reserved/committed and spot instances cost far less than on-demand for the right workloads.
Storage tiers	Move cold data to cheaper archival tiers; set lifecycle policies.
Caching & CDN	Serve from cache/CDN to cut compute and egress (Chapter 8).
Managed vs self-run	Trade a higher sticker price for lower operational cost — or vice versa.

49.4 The trade-off

KEY INSIGHT — COST IS ANOTHER DIMENSION OF DESIGN

Every reliability and performance choice has a price: more nines (Chapter 34), more replicas (Chapter 16), and multi-region (Chapter 48) all cost more. The goal isn't the *cheapest* system but the **right balance** — enough performance and reliability for the business at a defensible cost. Measure cost per request/user, attribute spend to teams, and revisit as scale changes. What's cheap at small scale can dominate the bill at large scale, and vice versa.

Common mistakes

WATCH OUT

- Over-provisioning for peak and running it 24/7 (no autoscaling).
- Ignoring egress and cross-region transfer costs.
- Leaving idle/orphaned resources and forgotten environments running.
- Optimizing cost so hard that reliability or performance suffers.
- No cost visibility or ownership (nobody watches the bill).

Interview questions

1. Why is cost a design constraint?
2. What are the main cloud cost drivers?
3. Which levers reduce cost, and what are their trade-offs?
4. How do you balance cost against reliability and performance?

Hands-on exercises

1. List the cost drivers for a photo-sharing app and rank them.
2. Decide on-demand, reserved, or spot for: a steady baseline, a batch job, spiky traffic.
3. Propose a storage-tiering policy for old user uploads.
4. Estimate the egress cost impact of serving media without a CDN.

Mini project

CUT A BILL IN HALF

Take a small always-on deployment and reduce its cost: add autoscaling so it scales down off-peak, right-size the instances, move cold data to a cheaper storage tier, and put static assets behind a CDN. Estimate before/after monthly cost and note what you traded (e.g. slightly slower cold-tier reads).

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE COST MODEL

Build a cost model: compute (autoscaled from your Chapter 5 numbers), storage (hot feed data vs archived old posts/media on cheaper tiers), and egress (media served via CDN to slash it). Choose reserved capacity for steady baseline and spot for batch jobs (feed recomputation), and set a target cost per active user.

Chapter summary

- Cost is a first-class **design constraint** — design to a target cost per request/user.
- Main drivers: **compute, storage, data egress, managed services, and idle resources**.
- Levers: **right-sizing, autoscaling, pricing models, storage tiers, caching/CDN**.
- Balance cost against **reliability and performance** — cheapest isn't the goal; the right balance is.

CHAPTER 50

Observability

Part VIII · Infrastructure & Operations — logs, metrics, and traces.

LEARNING OBJECTIVES

- Distinguish monitoring from observability.
- Explain the three pillars: logs, metrics, and traces.
- Explain distributed tracing across services.
- Explain alerting on symptoms and the role of SLOs.

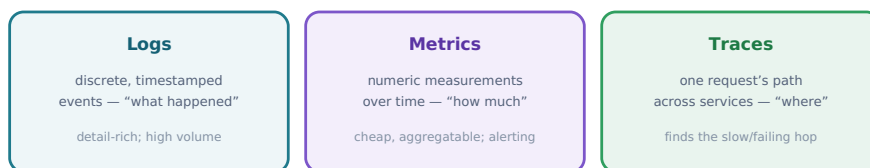
REAL-WORLD ANALOGY — A HOSPITAL'S INSTRUMENTS

A hospital doesn't guess a patient's condition — it watches vital-sign **monitors** (metrics), keeps a detailed **chart** of events (logs), and can trace a symptom through the body's systems (traces). Together these let doctors both spot trouble and diagnose *why*. Observability gives your system the same instruments.

50.1 The problem

Distributed systems fail in complex, partial ways (Chapter 19), and a request may cross many services (Chapters 29–30). When something breaks, you can't debug what you can't see. **Monitoring** tells you *whether* something is wrong (known questions, dashboards, alerts); **observability** is the property that lets you ask *why* — to explore new questions about a system's behavior from its outputs.

50.2 The three pillars



Pillar	What it is & best for
Logs	Discrete, timestamped event records. Rich detail for debugging a specific event; high volume, so sample/structure them.
Metrics	Numeric time-series (latency p99, error rate, CPU). Cheap and aggregatable — ideal for dashboards and alerts (Chapter 5).
Traces	The path of a single request across services, with timing per hop. Pinpoints <i>where</i> latency or failure occurs.

50.3 Distributed tracing

KEY INSIGHT — A TRACE ID STITCHES THE STORY TOGETHER

When one request fans out across many services, a **distributed trace** follows it end to end by attaching a **correlation/trace ID** that propagates through every call (the idea foreshadowed in Chapters 32 and 38). Each service records a **span** (its slice of work and timing), and together they show exactly which hop was slow or failed. Without it, debugging a microservices request is guesswork; with it, the answer is visible.

50.4 Alerting and SLOs

Alert on **symptoms users feel** — high error rate, high latency, low success — not on every internal metric, or you drown in noise and alert fatigue. Tie alerts to your **SLOs** and **error budgets** (Chapter 34): page a human when the user-facing objective is at risk. Good observability closes the loop: detect (metrics/alerts) → investigate (traces) → explain (logs).

Common mistakes

WATCH OUT

- Logging everything with no structure or sampling — unusable and expensive.
- No distributed tracing, leaving microservices requests undebugged.
- Alerting on causes/noise instead of user-facing symptoms (alert fatigue).
- Metrics without percentiles (averages hide tail latency, Chapter 5).
- Treating dashboards (monitoring) as a substitute for explorable observability.

Interview questions

1. Monitoring vs observability?
2. Describe logs, metrics, and traces and when to use each.
3. How does distributed tracing work across services?
4. What should you alert on, and how do SLOs guide alerting?

Hands-on exercises

1. For a bug “checkout is slow,” say which pillar you’d use at each step.
2. Add a trace/correlation ID that propagates across two services.
3. Define three symptom-based alerts tied to SLOs.
4. Explain why you’d track p99 latency, not the average.

Mini project

INSTRUMENT A TWO-SERVICE REQUEST

Add structured logs, metrics (request rate, error rate, p99 latency), and distributed tracing to two services that call each other. Propagate a trace ID, view a full trace, and

inject a slowdown in the downstream service — then use the trace to locate it and a metric-based alert to detect it.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE OBSERVABILITY STACK

Instrument every service with structured logs, RED metrics (Rate, Errors, Duration), and distributed tracing with propagated trace IDs (essential for your event-driven flows, Chapter 32, and sagas, Chapter 38). Define symptom-based alerts tied to your SLOs (Chapter 34), and build dashboards for feed latency, error rates, and queue depth.

Chapter summary

- **Monitoring** answers “is it broken?”; **observability** lets you ask **why**.
- Three pillars: **logs** (events/detail), **metrics** (numeric time-series/alerting), **traces** (a request across services).
- **Distributed tracing** uses a propagated **trace ID** and per-service **spans** to find the slow/failing hop.
- Alert on **user-facing symptoms** tied to **SLOs**, not noisy internal causes.

COMING NEXT

Chapter 51 — Capacity Planning closes Part VIII: turning the back-of-the-envelope numbers from Chapter 5 into concrete provisioning, headroom, and scaling decisions. Then Part IX assembles everything into a design-interview method.